

Model View Controller

Wikipedia - Website

Main Idea: The main idea of the MVC article is explaining what an MVC is. MVC is a way to organize three different components, a view, model, and controller.

Key Details: Some details about the articles include a history section and another section explaining what the different parts of MVC work. Users are able to see a view, where they will interact with a controller. Then the controller sends commands to the model, which then updates the view, which the user sees. This architecture has been used to design World Wide Web applications.

Relevance: This is relevant to our project, because we are in charge of making all three parts. The view is equivalent to our react front end. All of the UI design would be the design of the view. However, it is important to note that in the article, the view does not accept input in Smalltalk-80. Therefore, in this project, the view would be similar to the WebObjects definition of view. The controller would most likely be the Django backend. The model is most definitely the S3 + Lambda configuration for the autograder. The main workflow captures the frontend receiving input (submission), which gets sent to the Django backend (for sending commands and input to model), and lastly the Autograders spits out the result back to the view.

Reflection:

Although we can go through one instance of how a user might use the website, it is important to note that there are multiple different ways that a user might interact with the frontend, that does not necessarily need to use the backend. Sure, when an instructor adds a course or a student to a course, or a student searches for a course, all of those relations are stored in a database and django then comes into play. However, to actually navigate the website of the page, django is not needed. However, there is a router in the front end that manually sends a command to switch the page. Would that count as a controller? What about the database? How would the database fit into the MVC architecture? I think I have a lot of questions about the relationship between MVC and our project, but I do have a surface level interpretation.

The Model View Controller Pattern - MVC Architecture & Frameworks

FreeCodeCamp - Website

Main Idea: The main idea of the article is to explain the MVC pattern in terms of website components. The goal is to break down the actual meaning of all the components in an

applied sense.

Key Details: The article breaks down what exactly the view, controller, and model mean. The model would be the backend, where all the logic lies for how the data is handled. The view is the GUI, and the controller is the bridge between the two. The author made an example cat clicker website and then applied all of the concepts aforementioned.

Relevance: The description of MVC in this article makes a lot more sense than my interpretation of the wikipedia article. In this sense, our Django backend is the brains of the operation by containing the logic to interact with the database, S3, and Lambda. Then all of that data (and the response) is given to the controller which then decides how that data is viewed. I think that controller would be the top level part of the TypeScript files. That part would be the script part of an html file that could interact with an API or make requests to django.

Reflection: At first when reading the wikipedia article, I did not know how MVC was supposed to delegate tasks in the development process. This article cleared that up and answered other questions I had. I also think that the autograder stuff + the file buckets on S3 is a part of the model.